Guide de Conception d'un Agent Métier Marketing

Fondements SMA, Négociation FIPA-ACL & Intégration au Noyau Central

Document de Cadrage Méthodologique et Technique pour l'Équipe Projet | Semestre Académique

Objectif Global du Projet

Votre équipe a pour mission de développer un **Agent Logiciel de Marketing**. Dans la simulation, votre agent est responsable de valoriser, promouvoir et échanger des **produits finis** avec les agents conçus par les autres équipes (acheteurs, usines de fabrication ou distributeurs).
Pour orchestrer ce marché, votre enseignant fournit un **Noyau central** (middleware / bus d'échange).
Ce document synthétise tout ce que vous devez impérativement maîtriser pour débuter sereinement : concepts fondamentaux, fonctionnement du noyau, stratégie marketing, code d'intégration et pièges à éviter.

1. Comprendre les Agents et les Systèmes Multi-Agents (SMA)

1.1 Qu'est-ce qu'un Agent logiciel ?

Contrairement à un script classique ou à une API passive qui ne s'exécute que lorsqu'on l'appelle explicitement, un **Agent Intelligent** est une entité logicielle autonome située dans un environnement dynamique, dotée de buts propres et capable d'agir de son propre chef.

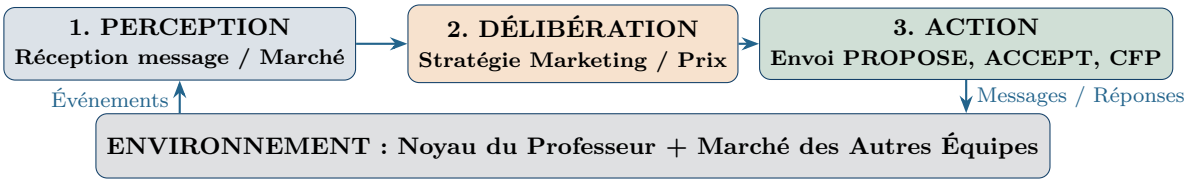
Critère	Programme Classique / Script	Agent Intelligent (SMA)
Contrôle	Passif : exécute l'ordre reçu sans délibérer.	Autonome : décide *si* et *comment* il répond.
Déclenchement	Appel de méthode synchrone externe.	Boucle autonome perpétuelle d'écoute et d'initiative.
Environnement	Entrées prévisibles et contrôlées.	Marché ouvert, concurrentiel et incertain.
Comportement	Déterministe (*Entrée* → *Sortie*).	Stratégique (*Perception* → *Délibération* → *Action*).

1.2 Les 4 Piliers d'un Agent (Modèle de Wooldridge & Jennings)

— **Autonomie :** L'agent contrôle son état interne, ses fonds et ses choix sans dépendre d'un humain.
— **Réactivité :** L'agent perçoit son environnement (offres reçues, alertes de marché) et réagit à temps.
— **Proactivité :** L'agent ne subit pas ; il prend l'initiative de démarcher des clients et de diffuser des offres.
— **Sociabilité :** L'agent communique et négocie via un protocole formel et structuré (actes de parole).

1.3 La Boucle Fondamentale : Perception → Délibération → Action

Le cœur d'exécution de l'agent tourne en continu selon trois étapes fondamentales :



1.4 Le Modèle BDI (Beliefs, Desires, Intentions)

Pour structurer la mémoire et le raisonnement de l'agent, le modèle d'architecture de référence est le modèle **BDI** :

— **Beliefs (Croyances / Connaissances) :** Ce que l'agent a en mémoire à l'instant t (stock disponible, trésorerie, historique des prix des concurrents, réputation des partenaires).
— **Desires (Désirs / Buts) :** Les finalités économiques (ex : écouler le stock au cours de la simulation en maximisant la marge nette).
— **Intentions (Plans engagés) :** L'action précise en cours d'exécution (ex : négocier une concession de 5% sur le lot #4).

2. Le Rôle du Noyau (« Kernel ») fourni par le Professeur

Le noyau fourni par votre enseignant ne concourt pas comme une équipe ; c'est **l'infrastructure d'arbitrage et de transport (Middleware)** du Système Multi-Agents.

2.1 Les 4 Fonctions Primordiales du Noyau

1. **Annuaire et Découverte (Directory Facilitator / Pages Jaunes) :** Registre officiel des agents connectés. Votre agent interroge l'annuaire pour découvrir qui vend et qui achète.
2. **Bus de Messagerie et Routage :** Chaque agent se connecte **uniquement au noyau**. Le noyau achemine les messages de façon centralisée et sécurisée.
3. **Arbitrage et Validation des Transactions (Registre Officiel) :** Si un contrat de vente est signé, le noyau valide que le vendeur possède les produits et que l'acheteur a les crédits nécessaires.
4. **Gestion du Temps (Clock / Rounds) :** La simulation progresse par tours synchronisés (*rounds* discrets cadencés) ou en flux continu asynchrone.

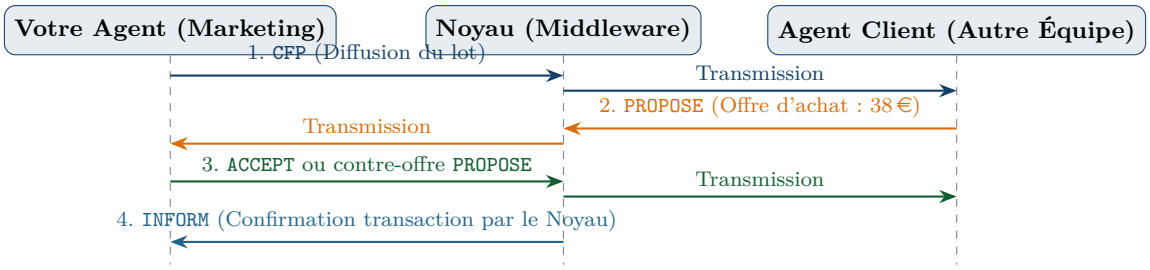
2.2 Le Protocole de Dialogue : Standard FIPA-ACL

Les échanges respectent la grammaire standard FIPA-ACL (*Agent Communication Language*), formulée en **JSON** :

Performative (Acte)	Intention du Message	Exemple d'Usage Marketing
`CFP` (*Call For Proposal*)	Appel d'offres / Sollicitation	« Avis au marché : qui souhaite acquérir 20 unités de Produit-A ? »
`PROPOSE`	Soumission d'une offre chiffrée	« Je propose 20 unités au tarif unitaire de 45 €. »
`ACCEPT_PROPOSAL`	Accord et conclusion du contrat	« Offre validée, nous confirmons la transaction. »
`REJECT_PROPOSAL`	Déclin de la proposition	« Prix inacceptable, proposition déclinée. »
`INFORM`	Transmission d'informations	« Notification : mise à jour de notre catalogue commercial. »

2.3 Protocole Type de Négociation : Contract Net Protocol (CNP)

Voici le schéma de négociation bilatérale le plus fréquent lors de la vente de produits finis :



Format Concret d'une Trame JSON FIPA-ACL

Tout message transitant par le noyau possède une structure standardisée :

```
{
  "sender": "Equipe_Marketing_3",
  "receiver": "Equipe_Acheteur_7",
  "conversation_id": "conv_lot_942",
  "performative": "PROPOSE",
  "content": {"product": "Module_Camera_HD", "quantity": 10, "unit_price": 42.50}
}
```

3. L'Agent Métier Marketing : Stratégies et Algorithmes Décisionnels

3.1 Missions Spécifiques de l'Agent Marketing

Dans la simulation, votre agent incarne la **direction commerciale** de votre équipe :

— **Positionnement Produit :** Calculer la valeur marchande des produits finis selon leurs caractéristiques.

— **Pricing Dynamique :** Ajuster les prix selon le coût de revient, la demande globale et l'échéance du temps.

— **Tactique de Concession :** Savoir accorder des rabais calculés pour conclure sans brader la rentabilité.

— **Veille Concurrentielle :** Consigner l'historique des transactions pour adapter ses marges aux autres équipes.

3.2 Modèle Mathématique de Tarification Dynamique Temporelle

Soit C le coût unitaire de fabrication, $P_{\max}$ le prix d'ouverture désiré, et $P_{\min}$ le prix plancher en dessous duquel l'équipe vend à perte ($P_{\min} > C$). Au tour ou temps $t \in [0, T]$, le prix cible $P(t)$ est calculé par :

$$P(t) = P_{\min} + (P_{\max} - P_{\min}) \cdot \left(1 - \frac{t}{T}\right)^{\beta} \tag{1}$$

— $\beta > 1$ **(Stratégie Concessionnaire) :** Baisse rapide des prix dès le début pour liquider les stocks.

— $\beta = 1$ **(Stratégie Linéaire) :** Concession régulière et constante au fil des tours.

— $0 < \beta < 1$ **(Stratégie Tenace / Écrémage) :** Maintien de prix élevés le plus longtemps possible.

3.3 Matrice Décisionnelle Face à une Offre Reçue

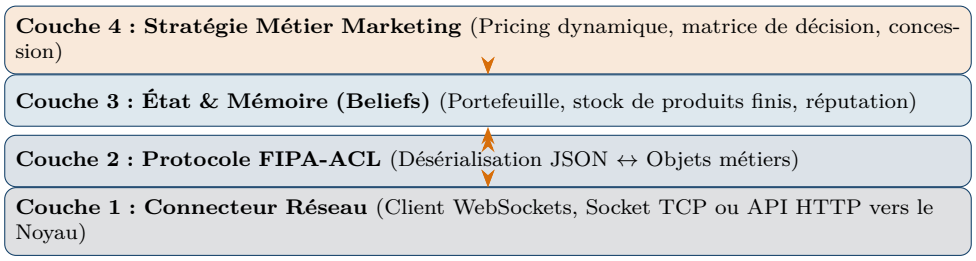
Quand un agent client vous soumet une offre de prix P_{offre}, la délibération applique cette règle formelle :

$$\text{Décision}(P_{\text{offre}}) = \begin{cases} \texttt{ACCEPT_PROPOSAL} & \text{si } P_{\text{offre}} \geq P(t) \\ \texttt{PROPOSE}\left(\frac{P(t)+P_{\text{offre}}}{2}\right) & \text{si } P_{\min} \leq P_{\text{offre}} < P(t) \quad \text{(Contre-offre de compromis)} \\ \texttt{REJECT_PROPOSAL} & \text{si } P_{\text{offre}} < P_{\min} \end{cases} \tag{2}$$

4. Architecture Logicielle & Guide d'Intégration au Noyau

4.1 Architecture Modulaire en 4 Couches

Pour rester sereins et agiles face aux évolutions du code du professeur, découpez votre agent en 4 couches hermétiques :



4.2 Checklist Technique : Les questions à poser impérativement au professeur

Checklist d'Intégration Réseau

[ ] **Moyen de transport :** WebSockets, Sockets TCP brutes, API REST HTTP, bus MQTT ou wrapper Python fourni ?

[ ] **Schéma des données :** Clés JSON exactes (`"performative"`, `"sender"`, `"receiver"`, `"content"`) ?

[ ] **Handshake :** Quel message exact envoyer pour s'enregistrer auprès de l'annuaire au lancement ?

[ ] **Cadencement temporel :** Simulation par tours (*rounds*) discrets ou temps réel asynchrone ?

[ ] **Structure d'un produit :** Quels attributs définissent un produit final (ID, qualité, coût unitaire) ?

5. Implémentation de Référence (Template Python Modulaire)

Ce squelette Python complet illustre la séparation absolue entre la logique de décision marketing et la tuyauterie réseau :

```python
import json

class MarketingStrategy:
    """Couche 4 (Metier) : Pricing dynamique et tactique de negociation."""
    def __init__(self, cost_price, min_margin=1.15, max_margin=1.70):
        self.cost = cost_price
        self.p_min = cost_price * min_margin      # Prix plancher absolu (rentabilite)
        self.p_max = cost_price * max_margin      # Prix d'ouverture ideal (forte marge)

    def compute_target_price(self, progress_ratio, beta=1.5):
        """Concession temporelle selon l'avancement du round (0.0 a 1.0)."""
        return self.p_min + (self.p_max - self.p_min) * ((1.0 - progress_ratio) ** beta)

    def evaluate_offer(self, offered_price, progress_ratio):
        target = self.compute_target_price(progress_ratio)
        if offered_price >= target:
            return "ACCEPT_PROPOSAL", offered_price
        elif offered_price >= self.p_min:
            counter_offer = round((target + offered_price) / 2.0, 2)
            return "PROPOSE", counter_offer
        return "REJECT_PROPOSAL", None

class MarketingAgent:
    """Couches 2 & 3 : Gestion de l'etat interne et reaction aux messages."""
    def __init__(self, agent_id, team_id, connector):
        self.agent_id = agent_id
        self.team_id = team_id
        self.connector = connector
        self.strategy = MarketingStrategy(cost_price=50.0)
        self.inventory = {}
        self.negotiation_history = []

    def register_to_kernel(self):
        """Enregistrement officiel aupres du noyau."""
        payload = {"performative": "REGISTER", "sender": self.agent_id, "team": self.team_id}
        self.connector.send(json.dumps(payload))

    def on_message_received(self, raw_json):
        """Boucle fondamentale : Perception -> Deliberation -> Action"""
        msg = json.loads(raw_json)
        performative = msg.get("performative")
        sender = msg.get("sender")
        content = msg.get("content", {})

        if performative == "CFP":
            # Reponse proactive a un appel d'offres entrant
            opening_price = self.strategy.p_max
            reply = {
                "sender": self.agent_id,
                "receiver": sender,
                "performative": "PROPOSE",
                "content": {"product_id": content.get("product_id"), "price": opening_price}
            }
            self.connector.send(json.dumps(reply))

        elif performative == "PROPOSE":
            # Evaluation d'une proposition de prix recue
            decision, price = self.strategy.evaluate_offer(content.get("price", 0),
                progress_ratio=0.5)
            reply = {
                "sender": self.agent_id,
                "receiver": sender,
                "performative": decision,
                "content": {"price": price} if price else {}
            }
            self.connector.send(json.dumps(reply))

        elif performative == "INFORM":
            # Confirmation d'une transaction executee par le noyau
            print(f"[{self.agent_id}] Transaction confirmee par le noyau : {content}")
```

Comment Exécuter ce Code dès Aujourd'hui ?

Vous n'avez pas besoin d'attendre le serveur du professeur pour tester ! Il suffit d'écrire une fausse classe `connector` avec une méthode `send(msg)` qui affiche le message à l'écran, et d'appeler manuellement `on_message_received(json_test)`.

6. Feuille de Route pour l'Équipe Projet

Pour avancer méthodiquement sans vous bloquer mutuellement, appliquez ce découpage en 4 phases :

1. **Sprint 1 : L'Agent Squelette (Dummy Agent)**
   - Mettre en place le dépôt Git et l'architecture en 4 couches.
   - Écrire le connecteur réseau minimal : connexion au noyau, envoi du message `REGISTER`, et confirmation des messages reçus (« ping/pong »).
2. **Sprint 2 : Le Simulateur Factice (« Mock Agent » local)**
   - Coder un petit script Python simulant un agent client acheteur sur votre machine locale.
   - **Avantage clé :** Permet à l'équipe de coder et tester la négociation sans attendre la disponibilité du noyau du professeur !
3. **Sprint 3 : Implémentation du Cœur Marketing**
   - Développer la fonction de prix dynamique et la gestion des concessions.
   - Mémoriser l'historique des prix des autres équipes pour optimiser les marges.
4. **Sprint 4 : Intégration Inter-Équipes & Résilience**
   - Déploiement grandeur nature avec les autres équipes sur le réseau universitaire.
   - Gestion robuste des anomalies (timeouts, messages mal formés, déconnexions).

Conseil Pratique : Répartition des Rôles au sein de l'Équipe

- **Étudiant 1 (Lead Réseau & Noyau) :** Connecteur (Sockets/WebSockets), sérialisation JSON, boucle d'écoute.
- **Étudiant 2 (Lead Métier Marketing) :** Algorithmes de pricing, matrice de décision, stratégies de concession.
- **Étudiant 3 (Lead QA / Simulation) :** Script de simulation factice (Mock), tests unitaires, validation des échanges.

7. Les 5 Pièges Fréquents en Projet SMA & Leurs Solutions

Attention aux Erreurs Classiques d'Étudiants

1. **Le Blocage Synchrone (Deadlock) :** Ne faites jamais un `socket.recv()` sans timeout au milieu de votre logique métier. Si un agent adverse plante, votre agent sera gelé. *Solution : Utilisez des écoutes asynchrones ou des threads séparés.*
2. **La Boucle Infinie de Négociation (Ping-Pong) :** Deux agents qui se renvoient sans cesse des contre-offres sans progresser font planter la simulation. *Solution : Ajoutez un compteur d'échanges (ex : max 3 tours de négociation par transaction).*
3. **La Vente à Perte par Concession Excessive :** Si vous concédez trop vite sous la panique du temps, vous vendrez en dessous de votre coût de revient C. *Solution : Imposez une barrière matérielle inviolable dans le code :* `assert price >= p_min`.
4. **Le Double Engagement de Stock :** Proposer le même lot de produits à deux acheteurs simultanés conduit à un échec d'arbitrage. *Solution : Réservez temporairement le stock lors de l'émission d'une proposition (*`reserved_stock`*).*
5. **Attendre le Noyau du Professeur :** L'erreur la plus coûteuse est de rester passif. *Solution : Développez immédiatement votre stratégie marketing en local avec des tests unitaires.*